

The Causality Game

Felix Mohr
Universidad de La Sabana
felix.mohr@unisabana.edu.co

Sergio Amortegui
Universidad de La Sabana
 sergioamar@unisabana.edu.co

Abstract

Causal inference seeks to uncover causal relationships across many domains including natural sciences, economics, and education. A core activity in causal inference is active intervention, without which reliable conclusions are often impossible. Since real-world interventions are costly, there is strong interest in agents that intelligently design and optimize interventions—such as experiments or treatments—balancing insight and experimental cost. Surprisingly, most benchmarks do not evaluate this ability. Instead, they score atomic agents that infer causality from a single dataset containing some interventions. We introduce **The Causality Game**, a benchmarking concept where agents actively choose interventions and are measured in both the quality of causal inferences and the efficiency of their discovery. We provide a Python package, `thecausalitygame`, which implements the framework in an extensible manner while including standard components to generate causal inference tasks of diverse types and over 50 built-in tasks for immediate use.

1 Introduction

[illegible]

A crucial activity in causal inference as a discipline is the active design of interventions [? ? ?]. Without some exceptions, it is not generally possible to conclude on causal relationships without systematic interventions [?], e.g., in the form of treatments or systematic exploration of an environment. Since the execution of such experiments is often very costly in practice, properly designing and executing them is arguably just as important as the final inference activity. In fact, without sufficient information from well-design experiments, no inference might be possible.

Despite its importance, this interactive dimension is almost entirely absent **SA: We are aware of** from existing benchmarks. Frameworks for causal graph discovery [? ? ? ?], symbolic regression [? ? ?], and treatment effect estimation [? ? ? ? ? ? ? ? ? ?] almost exclusively evaluate models on a fixed dataset, scoring only the final outcome rather than *how the knowledge was acquired*. The only known interactive benchmark, CausalWorld [?], focuses on robotic control and does not *explicitly* evaluate the correctness of causal reasoning itself. Thus, there is currently no rigorous framework for benchmarking *agent-based causal inference* as a problem of strategic experimental design.

To close this gap, we introduce **The Causality Game**, a turn-based, agent-centric causal inference framework in which players actively select interventions, gather observations, and decide when to stop and return a causal conclusion. As argued by [?], the distinction between causal graph discovery and treatment effect estimation is artificial, in particular since advanced treatment effect estimation techniques might solve causal graph discovery as a sub-task. To **The Causality Game**, these are simply different *missions* with respective *result metrics*, e.g., F1 or SHD for graph discovery, and PEHE for treatment effect estimation. Unlike static benchmarks, **The Causality Game** jointly evaluates (i) the *quality of causal conclusions* and (ii) the *efficiency of the experimental strategy*. The accompanying Python package `thecausalitygame` provides a rigorously modular implementation, enabling

pluggable agents, structural causal models (SCMs), behavior and result metrics, stopping functions, and fully seedable execution for reproducible comparison across agents and settings.

Contributions. This work introduces:

- A formalized *agent-based causal inference game* that evaluates not only *what* an agent concludes, but *how intelligently* it gathers information.
- A modular, extensible *system architecture* implemented in the `causalitygame`, supporting dynamic SCM generation from datasets, Bayesian networks, and synthetic rules.
- A principled framework for multi-mission evaluation, covering DAG recovery, CATE estimation, SCM reconstruction, and beyond.
- A fully seedable execution model enabling *fair, reproducible, and comparable* evaluation of heterogeneous causal agents.

By reframing causal inference as a strategic experimental process rather than a passive estimation task, **The Causality Game** enables evaluation of the next generation of *interactive causal agents*—those that do not merely estimate causality, but *actively learn it*.

2 The Causality Game — Formal Framework

The Causality Game is not a metaphor but a formal interactive system for causal discovery and reasoning grounded in structural causal models (SCMs). It represents a controlled, turn-based environment in which an agent performs interventions, observes results, and progressively uncovers causal relationships among a finite set of variables. The following formulation describes the game as a mathematical object, providing the foundation upon which its software implementation is built.

2.1 Definition and Core Components

A single game instance is defined by the tuple

$$\langle M, \mathcal{X}_H \sqcup \mathcal{X}_M \sqcup \mathcal{X}_C, s_0, \phi, g, p_r, p_b \rangle,$$

where:

- M is the underlying structural causal model that governs the world’s data-generating process.
- $\mathcal{X}_H, \mathcal{X}_M, \mathcal{X}_C$ partition $\mathcal{X}$, determining which variables are hidden, observable, or intervenable.
- $s_0 \in \mathcal{S}$ represents the starting data context, typically empty or initialized with baseline observations under the null intervention $\perp$.
- $\phi : \mathcal{R} \rightarrow \{0, 1\}$ is a stopping function that enforces runtime limits such as sample count, memory, or time budget.
- g defines the mission — the causal question to be solved.
- $p_r : \mathcal{O} \rightarrow \mathbb{R}$ scores the correctness of the final answer.
- $p_b : \mathcal{R} \rightarrow \mathbb{R}$ measures the efficiency of the agent’s experimentation behavior.

The agent does not directly observe M or any $\mathcal{X}_H$; instead, it interacts through interventions on $\mathcal{X}_C$ and receives measured outcomes from $\mathcal{X}_M \sqcup \mathcal{X}_C$.

2.2 Extended Structural Causal Model

Formally, the SCM as the foundation of the game, is defined as

$$M = \langle \mathcal{X}, \{f_i\}, \{\eta_i\} \rangle,$$

where $\mathcal{X}$ is the set of variables, f_i are deterministic structural functions, and η_i are exogenous noise terms. In the context of **The Causality Game**, each variable also has a declared *access mode*:

$$\mathcal{X}_H \sqcup \mathcal{X}_M \sqcup \mathcal{X}_C = \mathcal{X}.$$

This configuration explicitly controls which nodes the agent may observe or intervene upon. The SCM thus defines both the ground-truth causal structure and the constraints under which agents can interact with it, reflecting real-world experimental limitations.

2.3 Environment and Experiment Execution

The environment is a deterministic causal oracle induced by the extended SCM. It does not maintain any mutable internal “state.” Instead, it responds to each agent action by *functionally generating new data* and returning it to the agent. All memory of past interactions is externalized into the **Transcript**, which is maintained by the **The Causality Game** orchestrator, not by the environment itself.

Interventions. An intervention is a partial assignment over controllable variables:

$$t : X_i \in \mathcal{X}_C \mapsto v \in \text{domain}(X_i) \cup \{\text{None}\}, \quad t \in \mathcal{I},$$

where $t(X_i) = \text{None}$ means that X_i is left to follow its natural SCM equation. The empty intervention $\perp$ corresponds to purely observational sampling.

Experiments. Each *environment action* is a finite experiment request:

$$e : \mathcal{I} \rightarrow \mathbb{N},$$

meaning that for each intervention t with $e(t) = n > 0$, the environment returns n new i.i.d. samples generated under that interventional condition. There is no deferred scheduling: each experiment is executed immediately in the same round it is requested.

Interaction Protocol (inform $\rightarrow$ act). The agent never receives a raw environment state. Instead, the interaction in each round proceeds in two stages:

1. **inform:** The agent is given the newly generated samples from its previous experiment, restricted to $\mathcal{X}_M \cup \mathcal{X}_C$, together with a structured feedback object (e.g. partial scoring about how well its current hypothesis aligns with the mission).
2. **act:** The agent is then given two structured DTOs:
 - a `RoundInformation` object summarizing the current budget status, mission description, etc.,
 - an `AvailableActions` object enumerating all legal next experiment configurations plus the special terminal option *stop with answer*.

It must then choose either another experiment $e \in \mathcal{T}$ or stop and submit a final answer.

2.4 Environment Determinism and Reproducibility

The agent’s experience is not merely stochastic — it is *seed-deterministic*. The initialization seed fixes the entire sample-generation randomness of the environment. Thus, two independent agents that issue the same sequence of actions will observe *bitwise identical outcomes*. This ensures **perfect fairness and reproducibility** across agents and research groups — a critical requirement for benchmark integrity.

2.5 Start and Stop Conditions

The temporal extent of a run is controlled by two mechanisms:

- an explicit **initial state** $s_0 \in \mathcal{S}$, which determines what data (if any) the agent sees before taking its first action,
- an external **stopping function** $\phi : \mathcal{R} \rightarrow \{0, 1\}$, which may forcibly terminate the game if resource constraints are exceeded (e.g. too many interventions or samples have been requested).

Crucially, the agent is always free to stop *voluntarily* at any round by issuing a *stop with answer* action, returning some $y \in \mathcal{O}$. If ϕ triggers first, the game ends immediately *without* returning an answer — resulting in a zero or penalty score under p_r .

Scientific motivation. This design mirrors real experimental science: if a researcher exhausts funding before returning a final conclusion, there is *no result to evaluate*. Thus, **The Causality Game** formalizes causal inference not only as a problem of correctness — but of disciplined experimental planning under pressure.

This reflects real-world constraints: experimenters may choose to conclude early, but if they exceed allowable resources, the process is terminated before any formal conclusion can be submitted.

2.6 Capabilities and Limits

Within each run, the agent:

- can perform any legal intervention $t \in \mathcal{I}$ on $\mathcal{X}_C$,
- can request n samples per intervention via experiments,
- receives only observable outcomes $\mathcal{X}_M$,
- and may stop at any point with an answer submission.

It cannot:

- access or view latent variables $\mathcal{X}_H$,
- modify the true M or observe its internal structure,
- or bypass the stopping rule ϕ once triggered.

This strict separation between what the agent *can* and *cannot* know enforces an epistemically valid experimental process.

2.7 Objective

A *player agent* interacts with the game instance by selecting actions in discrete turns to generate a run

$$r = (s_0, a_0, s_1, a_1, \dots, a_{T-1}, s_T), \quad r \in \mathcal{R}.$$

Its goal is to produce a final answer $y \in \mathcal{O}$, by issuing a *stop with answer* action before termination, such that $p_r(y)$ is maximized while incurring minimal cost under $p_b(r)$.

Formally, a possible multi-objective evaluation to maximize and rank agents:

$$\text{score}(r) = (p_r(y), -p_b(r)), \quad \text{Pareto-optimality is well-defined.}$$

However. No scalarization is imposed at the framework level — users may rank agents by convex combination, lexicographic ordering, or full Pareto frontier analysis.

2.8 Narrative Summary

The Causality Game thus formalizes the process of causal discovery as an interactive and bounded experimentation problem. At each round, the agent receives structured contextual information, selects from permissible actions, and shapes the data it will later use to infer causal structure. Through this formulation, causal learning becomes a reproducible, quantitative, and comparable process — bridging the conceptual gap between theoretical causal reasoning and experimental design.

3 Structural Causal Model (SCM)

At the core of every **The Causality Game** instance is the **Structural Causal Model (SCM)**, which defines the causal laws of the environment. The SCM is the *only source of ground-truth causality* — every observation, every experiment outcome, and every causal dependency observed by the agent originates here.

While its formal definition was introduced in Section 3.1, we now expand it from the viewpoint of the **runtime engine** — where it becomes an *active, queryable data-generating process*.

Executable SCM. In the environment, the SCM is not a static object but a *callable simulation oracle*:

$$M.\text{sample} : t \longrightarrow \mathcal{D},$$

which generates samples from $\mathcal{X}_M$ variables under a specified intervention on $\mathcal{X}_C$ — always respecting the DAG and functional dependencies encoded in $\{f_i\}$.

Formally:

$$\mathcal{D} = \left\{ x^{(j)} \in \text{dom}(\mathcal{X}_M) \mid x^{(j)} \sim M \text{ under } \text{do}(t) \right\}.$$

Strict purity. The SCM is **stateless, reproducible, and seed-bound**. Executing the same intervention twice yields identical outputs at the *same positions in the random sample stream* — ensured via the initialization seed. This enables *perfect inter-agent comparability*, even across machines.

3.1 SCMs from Factual Datasets

Datasets do not induce unique SCMs but are *one* ingredient for an SCM, complemented by other components. Importantly, different outcome generators or even the same outcome generator initialized with different random seeds will lead to different SCMs [?].

thecaualitygame ships three datasets from which SCMs can be derived, which are summarized in Table 1. These are the datasets IHDP [?], Jobs [?], and Twins [?] commonly used in causal inference papers. Despite the justified critics of some over-use of these datasets [?], they *do* provided added value over merely observational datasets, which bears the chance to develop arguably more realistic outcome generators than those commonly considered. Then, on a more positive note, packages like thecaualitygame can contribute to make such more realistic outcome generators available to a broad audience and help mitigate the problem of semantically flawed de-facto standard benchmarks.

name	# vars	# cat vars	# numeric vars	dataset size	min/avg/max domain size	outcome type	has counterfactuals
IHDP	30	24	6	747	2/40/747	numerical	False
Jobs	9	5	4	722	2/112/524	numerical	False
Twins	50	44	6	64240	2/8/58	categorical	True

Table 1: Properties of the three datasets shipped with thecaualitygame to derive SCMs from data.

3.2 SCMs from Standard Bayesian Networks

We compiled all 25 Bayesian networks provided by [?] into SCMs. These networks range in complexity between only a hand full to over a thousand variables and different connection complexities between them. An overview of the SCM properties is shown in Tab. 2. All variables in these networks are categorical.

name	# vars	# edges	# roots	# leaves	max #pa	max #ch	min/avg/max dom.s	name	# vars	# edges	# roots	# leaves	max #pa	max #ch	min/avg/max dom.s
alarm	37	46	12	11	4	5	2/3/4	mildew	35	46	16	1	3	3	3/18/100
andes	223	338	89	25	6	12	2/2/2	munin	1041	1397	259	183	3	69	2/5/21
asia	8	8	2	2	2	2	2/2/2	munin	186	273	34	31	3	15	2/5/21
barley	48	84	10	8	4	5	2/9/67	munin_sn2	1003	1244	249	182	3	30	2/5/21
cancer	5	4	2	2	2	2	2/2/2	munin_sn3	1041	1306	262	186	3	66	2/5/21
child	20	25	1	7	2	7	2/3/6	munin_sn4	1038	1388	259	180	3	66	2/5/21
diabetes	413	602	76	2	2	24	3/11/21	pathfinder	109	195	1	77	5	106	2/4/63
earthquake	5	4	2	2	2	2	2/2/2	pigs	441	592	145	141	2	39	3/3/3
hailfinder	56	66	17	13	4	16	2/4/11	sachs	11	17	2	4	3	6	3/3/3
hepar2	70	123	9	41	6	17	2/2/4	survey	6	6	2	1	2	2	2/2/3
insurance	27	52	2	6	3	7	2/3/5	water	32	66	8	8	5	3	3/4/4
link	724	1125	184	133	3	14	2/3/4	win95pts	76	112	34	16	7	10	2/2/2

Table 2: Properties of the SCMs derived from fully discrete Bayesian networks.

3.3 SCMs from Physical Laws

Even though many recent benchmarks seek to compare performance on complex real world scenarios like traffic [?], climate change [?] or biological processes [?], the problem of those is that ground truth SCMs are not available or arguably secured. Even though there is clearly an interest in assessing causal inference methods on highly complex problems at *some* point, such methods should arguably first qualify by high performance on dozens or hundreds of much simpler problems where the ground truth is known.

A specifically suitable type of task is to establish “laws” in physics from data [? ? ?]. The advantage of such laws is that, even though they usually come in the form of *algebraic* equations without necessarily a direction of causation, many laws *do* exhibit a direction of causation. For example, Ohm’s law $V = IR$ actually poses a *causal* connection that can be directly used in an SCM equation for V (but not to rearrange for R).

thecaualitygame comes with 20 SCM definitions from the area of laws in physics, which are summarized in Table 3. These are basic equations with just a single equation, but they can be combined to form SCMs where the agent must learn multiple (possibly interacting) relationships. However, we took care that the direction in all physical SCMs was compatible with a valid causal interpretation of the respective equation formula. While

the number of tasks is still smaller than that in SRBench [?], in thecausalitygame they can be addressed (i) interactively and (ii) for different tasks than only symbolic regression. The interactive part is very important since it enables an active learning environment in which players can compete on who needs less (intelligently chosen) observations to uncover the relationship in the equation; the above-mentioned benchmarks confront each player with the same (often quite large) amount of data.

name	# vars	# edges	# roots	max #pa	name	# vars	# edges	# roots	max #pa
ac_electrical_power	4	3	3	3	newton_2nd_law	3	2	2	2
brewsters_angle	3	2	2	2	ohms_law_1	3	2	2	2
capacitor	3	2	2	2	ohms_law_2	3	2	2	2
circular_motion	5	4	3	2	ph	2	1	1	1
gravitational_force	5	4	4	4	radioactive_decay	4	3	3	3
hookes_law	3	2	2	2	rc_circuit	6	5	4	3
ideal_gas	5	4	4	4	refraction_snell	4	3	3	3
kepler_3	3	2	2	2	resonance_frequency	3	2	2	2
kinetic_energy	3	2	2	2	stefan_boltzmann	3	2	2	2
lensmaker	4	3	3	3	torricelli	3	2	2	2

Table 3: The 20 causally interpreted laws in thecausalitygame to define SCMs.

4 System Architecture — Instantiation of the Formal Game

The formal game defined in the previous section 2 is realized in thecausalitygame as a principled causal experimentation protocol — not as a loose software project. Its architecture is explicitly designed to preserve (i) the causal access restrictions, (ii) perfect reproducibility, and (iii) agent neutrality, while remaining fully modular and extensible.

4.1 Problem Instance as the Root Abstraction

In implementation, a *problem instance* is the root-level object that binds all components of a game together. It encapsulates exactly the tuple defined previously at the formal level:

$$\langle M, \mathcal{X}_H \sqcup \mathcal{X}_M \sqcup \mathcal{X}_C, s_0, \phi, g, p_r, p_b \rangle.$$

However — crucially — this is constructed *not in code directly*, but via a fully serializable **Spec configuration**. This ensures:

- **Perfect reproducibility:** any experiment can be saved and replayed bit-accurately.
- **Pluggability:** replacing the SCM, mission, or agent requires no change to the runtime engine.
- **Parallel evaluation:** multiple instances can be run concurrently, with isolated seeds and transcripts.

Every execution of **The Causality Game** begins by *loading a problem instance from Spec*, then passing it to the **Runtime**, which produces a **Transcript** — the immutable record of the agent’s interaction with the SCM.

What follows is a deep dive through each architectural layer — beginning with the role of Contracts, the foundation that makes **The Causality Game** not only a tool, but a *scientific protocol*.

4.2 Contracts — Enforcing Causal Boundaries

Every core component in **The Causality Game** (agent, SCM, mission, metric, hook, plot, etc.) is defined first by an abstract *contract*. A contract specifies *what a component is allowed to do* — never *how it must be implemented*. This mirrors scientific protocol design: an experimental instrument is defined by its *accessible interface*, not by its internal mechanics.

The agent contract, for example, can act and be informed, but cannot inspect the SCM. The SCM contract can generate interventional data but knows nothing about the agent’s beliefs. Result metrics see the ground truth; agents never do. This disciplines the entire system to respect causal epistemic validity — knowledge must be earned, not granted.

4.3 Specs — Declarative Instantiation of a Game Instance

A game instance is not created manually in Python code. Instead, every component is declared via a *Spec*: a fully serializable description of which class to use (via import path) and with which parameter configuration. A centralized registry instantiates these Specs at runtime — deterministically and seed-bound.

This makes **The Causality Game** *perfectly reproducible* and experiment definitions *immutably declarative*: a problem instance is not code, but data. Replacing the agent or SCM requires no change to the runtime engine — only to the Spec.

4.4 DTOs — Controlled Information Flow

During execution, every piece of information passed between components flows solely through *Data Transfer Objects (DTOs)* — strictly typed, immutable containers. They act as *information firewalls*, ensuring that no component receives more data than its contract permits.

For example, an agent receives only observable samples and round metadata — never latent variables or SCM internals. Metrics and plots may later access the full transcript and ground truth — but only after the game has ended.

DTOs thus preserve causal fairness even when evaluating extremely powerful or self-reflective agents.

4.5 Managers — Instrumentation and Enforcement

Managers regulate the execution of **The Causality Game** without influencing its causal logic. They do not assist the agent or modify the SCM; instead, they enforce constraints and expose observability.

BudgetManager charges resource costs (e.g. samples, rounds, time, memory) and triggers termination via ϕ when limits are exceeded — making experimentation economically meaningful.

HookManager listens to lifecycle events (e.g. `GAME_START`, `ROUND_END`, `GAME_END`) and triggers user-defined side-channel observers such as loggers or monitors. Hooks are strictly passive: they can record or visualize, but never intervene.

PlotManager generates visual artifacts post-hoc (e.g. at game or benchmark end), never visible to the agent during play, ensuring causal purity.

In short, managers provide accountability, observability, and control — never assistance or information leakage.

SA: This might be deleted.

4.6 SCM Construction Process

The Causality Game allows for the construction of an explicit, fully specified causal model comprising a directed acyclic graph (DAG), semantically typed variables, and structural equations with exogenous noise. This process ensures that every resulting SCM is identifiable, controllable, and suitable for scientifically valid causal experimentation.

4.6.1 DAG Topology and Structural Constraints

The causal graph is defined as a directed acyclic graph

$$\mathcal{G} = (\mathcal{X}, \mathcal{E}), \quad \mathcal{E} \subseteq \mathcal{X} \times \mathcal{X},$$

where an edge $(X_j \rightarrow X_i)$ implies direct causal influence via the structural equation of X_i . DAG generation is never arbitrary — it is constructed to enforce causal identifiability, appropriate difficulty, and benchmark realism.

A *strict topological ordering* is sampled such that all edges respect $j < i$, guaranteeing acyclicity and enabling forward-sampling during data generation. The DAG is aligned with the access partition

$$\mathcal{X}_H \sqcup \mathcal{X}_M \sqcup \mathcal{X}_C = \mathcal{X},$$

where latent variables may act as confounders, only measured variables are observable to the agent, and only controllable variables may be intervened on during gameplay. This partition is fixed once the SCM is instantiated and cannot change mid-run.

Causal complexity is explicitly controlled by bounding the in-degree of each node (e.g. $\deg^-(X_i) \leq d_{\max}$) and the maximum causal depth. This prevents trivial graphs and also avoids pathological, overly deep causal chains. Unlike most existing benchmarks, **The Causality Game** natively supports *hybrid* causal graphs containing both continuous and categorical nodes. Variable type proportions are user-controlled, enabling diverse families of causal structures.

Finally, while the DAG may encode arbitrarily rich parent sets, its structure is never leaked — the agent cannot inspect or query $\mathcal{E}$ directly. All knowledge arises solely through interventions and observed consequences.

4.6.2 Variable Definitions: Type, Domain, and Access

Once the graph structure is fixed, each variable $X_i \in \mathcal{X}$ is assigned a semantic specification determining how it participates in the SCM.

Each variable has a declared type

$$\text{type}(X_i) \in \{\text{categorical}, \text{continuous}\},$$

with proportions governed by a user-defined parameter p_C . Continuous variables operate on bounded real intervals, while categorical variables draw from a finite, user-configurable domain $\{c_1, \dots, c_K\}$. Domains are immutable — interventions may only assign values already in the domain, never extend it.

In parallel, each variable is assigned an access role:

$$\text{access}(X_i) \in \{\text{latent}, \text{observable}, \text{controllable}\}.$$

Latent variables never appear in any dataset, observable variables appear in all samples returned to the agent, and controllable variables may be directly set through interventions. This access role is contractual and strictly enforced throughout the game.

At this point, each variable has a clearly defined type, legal value range, position in the DAG, and epistemic status relative to the agent — forming a complete “vocabulary” for the SCM.

4.6.3 Structural Equations and Noise Generation

The final layer of construction defines how each variable’s value is *generated* during sampling. For each variable X_i with parent set $\text{Pa}(X_i)$, the SCM defines a structural assignment

$$X_i := f_i(\text{Pa}(X_i)) + \eta_i,$$

where f_i is a deterministic function and η_i is an exogenous noise term, independently drawn per variable.

In SCMs generated from scratch, f_i is sampled from a symbolic function grammar with configurable expressiveness — supporting operators such as addition, multiplication, exponentiation, logarithm, trigonometric functions, and bounded polynomial degrees. The function is generated exactly once at SCM creation time and remains immutable thereafter.

For categorical targets, class-specific scores are generated and normalized into a valid discrete distribution. Noise variables η_i are drawn from family-specific distributions (Gaussian, Laplace, uniform, Dirichlet, or degenerate Dirac for fixed treatment variables), with output values always clipped or transformed to remain in the declared domain of X_i .

This mechanism cleanly separates deterministic functional influence from stochastic exogenous uncertainty — allowing the SCM to express richly structured causal phenomena while remaining perfectly simulatable under interventions.

—

This completes the SCM construction process: a topologically valid, access-partitioned DAG equipped with semantically defined variables and precise structural equations — ready to serve as the causal ground truth for interactive experimentation in **The Causality Game**.

4.7 Missions — Declarative Definition of the Causal Task

A **Mission** is a declarative, contract-based specification that defines *what causal problem the agent is expected to solve*. It encodes the semantic nature of the task — for example, learning the full DAG, estimating the ATE or

CATE, or reconstructing structural mechanisms — but it does **not** evaluate correctness itself. Instead, a mission orchestrates *which output format* the agent must produce ($\mathcal{O}$) and *which evaluators apply* (i.e., which **result** and **behavior** metrics will later be invoked).

In other words, missions do not compute scores — they define the rules of the scientific game. They specify the expected answer representation and the evaluative lenses through which agent performance will be interpreted. This makes them a purely semantic layer — strictly framing *what “solving the task” means* without influencing the environment or agent logic.

Because missions are declared via Specs, switching from “full DAG discovery” to “ATE estimation” requires no modification to the agent or SCM — only a change in mission configuration. Missions therefore serve as the *semantic lens* through which causal inference is framed and evaluated in **The Causality Game**, keeping the system modular and scientifically expressive.

4.8 The Agent Perspective — Action, Observation, and Learning

From the environment’s point of view, the SCM is the causal world. From the agent’s point of view, the world is experienced only as a sequence of DTO-restricted observations and permissible action opportunities. The agent operates entirely under partial observability, without privileged access to the true SCM, while navigating unknown ground truth and being subject to explicit resource constraints.

Every agent conforming to the Agent contract implements two methods: `act(...)` and `inform(...)`. At the beginning of each round, the agent is invoked with `act(RoundInformation, AvailableActions)`, where it receives only legal meta-information and the currently admissible action space. It must then propose either a valid experiment or a terminal *stop with answer* action. After the environment executes that request, the agent is called with `inform(Samples, Feedback)`, receiving only the observable new samples and optional mission-defined feedback — never any internal representation of the SCM. Learning agents may use this information to update latent internal beliefs, but have no direct access to the transcript, budget counters, or ground truth.

This observational restriction is absolute. The agent never sees SCM equations f_i , noise variables η_i , latent variables $\mathcal{X}_H$, future reward signals, or the true causal structure. It operates solely on previously observed samples (restricted to $\mathcal{X}_M$), optional round-level metadata provided through DTOs, and the explicitly declared permissible action space.

At any time, the agent may choose to stop voluntarily by submitting a final answer $y \in \mathcal{O}$. This ends the game immediately. If it fails to do so before the stopping function ϕ triggers, termination occurs without an answer, and the result metric assigns a failure score.

Crucially, **The Causality Game** is architecturally neutral: it does not assume, privilege, or restrict agents to any particular paradigm. Agents may be symbolic search procedures, Bayesian experimental designers, deep learning models, reinforcement learners, or any hybrid thereof. All are subjected to the exact same causal interaction protocol — no agent is granted information outside legal DTOs, and no agent may bypass the constraints of the environment.

With the agent interface now fully stated, we proceed to define how **The Causality Game** evaluates performance — not only in terms of correctness of the final answer, but also in terms of the efficiency and prudence of the experimental process.

4.9 Result Validators — Ensuring Output-Format Legitimacy

A **Result Validator** is the gatekeeper that checks whether the agent’s submitted answer $y \in \mathcal{O}$ is *well-formed and admissible* — before any scoring is attempted. Its role is strictly *pre-evaluative*: it ensures that the output matches the mission-defined schema, datatype, and structural constraints.

This validation step prevents malformed or logically invalid outputs (e.g. ill-shaped tensors, non-DAG matrices, out-of-domain predictions) from being passed to the p_r . If the output is invalid, the validator can either reject it outright or return a canonical “invalid answer” token for scoring. Crucially — **the validator does not judge correctness** — it only ensures that the answer is structurally legitimate input for the result metric.

Once validation succeeds, the **result metric** takes over, comparing the validated output to the mission’s ground truth and assigning the actual correctness score.

4.10 Evaluation Objectives — Behavioral vs. Result Metrics

A defining principle of **The Causality Game** is that agents are *not judged solely on correctness*. Their performance is evaluated along two orthogonal dimensions: the quality of the causal answer they ultimately submit, and the efficiency with which they conducted the experimental process required to obtain it.

The first dimension is the **result metric** $p_r : \mathcal{O} \rightarrow \mathbb{R}$, which evaluates the agent’s final answer y after it voluntarily issues a stop-with-answer action. The score reflects how well y matches the ground truth, and may take the form of Structural Hamming Distance or F1 for DAG recovery, PEHE or mean-squared error for treatment effect estimation, or functional similarity or KL divergence for full SCM reconstruction. If the agent fails to return an answer before the stopping function ϕ is triggered, it receives a mission-dependent minimal score (e.g. 0 or $-\infty$).

The second dimension is the **behavior metric** $p_b : \mathcal{R} \rightarrow \mathbb{R}$, which evaluates the entire trajectory r rather than its final outcome. This reflects the agent’s experimental prudence — for example, by penalizing excessive sample usage, intervention count, or round duration via a task-specific cost model.

Crucially, **The Causality Game** does not combine these scores into a single scalar value. Instead, it returns the two-objective outcome

$$(p_r(y), p_b(r)),$$

allowing users to apply their preferred aggregation — such as convex combination, lexicographic ordering, or explicit Pareto frontier analysis.

By separating correctness from experimentation strategy, **The Causality Game** makes efficiency–accuracy trade-offs **formally measurable**. It distinguishes, for the first time at benchmark scale, between an agent that is perfectly accurate but recklessly expensive, and an agent that is slightly less accurate but vastly more efficient.

4.11 Serialization and Artifacts

A defining property of **The Causality Game** is that every execution is **perfectly reproducible, auditable, and replayable** — down to the exact sequence of interventions chosen, samples observed, and decisions made. This is made possible through strict serialization at the level of experimental definition and explicit artifact generation at runtime.

Because every game instance is declared entirely via Specs (cf. Sec. 4.3) and every SCM is executed under a fixed global seed, an experiment is serializable *before it even begins*. The full configuration — including SCM provenance, agent parameters and seeds, mission definitions, metric specifications, and budget or hook rules — is recorded deterministically.

At runtime, the system produces a **Transcript artifact**: a structured, DTO-encoded log that captures every intervention request, every sample revealed to the agent, every decision made (including the final answer, if any), and every applied cost or forced stopping event. This transcript is immutable, machine-parseable, human-inspectable, and replayable — it is the single source of truth for evaluation, visualization, and scientific auditing.

This architecture ensures that any experiment conducted in **The Causality Game** can be reproduced exactly across machines, audited rigorously for fairness or correctness, revisited with new evaluation tools, and safely shared as a benchmark asset — all without leaking ground-truth causal information prematurely.

With serialization guaranteed, we now turn to how **The Causality Game** scales this rigor to large-scale empirical evaluation — enabling the concurrent benchmarking of many agents, SCMs, or missions in parallel.

4.12 Runner and Parallel Execution

While a single **Game** represents one agent–SCM interaction, most real experimental scenarios require running many such games — for example, comparing agents across different SCMs, stress-testing a single agent over many seeds, or sweeping entire agent $\times$ SCM $\times$ mission grids. To support this, **The Causality Game** provides a dedicated orchestration layer called the **Runner**.

The Runner automates the execution of multiple game instances, either sequentially or in parallel, while guaranteeing full isolation: each run receives its own SCM instance (even if derived from the same Spec), its own agent and random seed, and its own transcript. No run may observe or influence another — preserving experimental purity even in large-scale or distributed evaluation settings.

Once execution finishes, the Runner performs systematic post-processing. It surfaces a unified collection of transcripts, aggregated metric evaluations, and — where specified — benchmark-level visualizations or Pareto

frontier analyses triggered by the PlotManager. In doing so, it does not modify or reinterpret the underlying causal game; it merely manages repetition, isolation, and aggregation.

4.13 A Scientific Protocol, Not Just Software

The architecture above is deliberately minimal, but precisely controlled. Every component is replaceable, but only through declared Specs. Every piece of information is visible only where epistemically valid. Every run is fully replayable. **The Causality Game** is therefore not a heuristic system, but a **formal causal experimentation protocol instantiated in software**.

With the architecture now formalized as an *executable causal protocol*, we are ready to define how it is used — the *experimental protocol* that governs evaluation, comparability, and scientific validity.

5 Experimental Protocol

With the formal game and its system architecture fully defined, we now turn to how experiments are **systematically designed, executed, and evaluated** using **The Causality Game**. This section establishes a **standardized scientific workflow** that enables fair, reproducible benchmarking of causal agents across missions, SCMs, and budget constraints.

5.1 Defining a Problem Instance

Every experiment begins by instantiating a **problem instance** via Specs (see Sec. 4.3). This consists of choosing:

- a **mission specification** (e.g. DAG discovery, CATE estimation, SCM reconstruction),
- a **ground-truth SCM** (from dataset, Bayesian network, or synthetic generator),
- **initial conditions** (s_0),
- **stopping conditions** (ϕ — typically a resource budget),
- one or more **evaluation metrics** (p_r, p_b).

This problem instance is fully declarative — once written, it may be executed indefinitely by any number of agents under *identical conditions*.

5.2 Selecting and Configuring Agents

Agents are likewise instantiated via Specs under the Agent contract. Each agent may have its own:

- seed,
- exploration strategy,
- allowable internal memory or policy class,
- explicit hyperparameters (e.g. max experiments per round).

No agent may modify the problem instance or SCM at runtime. All causal interaction is exclusively mediated through the Environment and DTO system.

5.3 Running the Game

The experiment proceeds by calling:

`Game.run()` $\longrightarrow$ `Transcript`.

This produces a **sealed interaction log** with:

- every action taken,
- every sample observed,
- the budget consumption trajectory,
- and the final stop decision (or forced termination).

When multiple agents or SCM seeds are evaluated, this step is typically orchestrated via the Runner (see Sec. 4.12).

5.4 Evaluating Performance

Evaluation is **post-hoc** — the agent is never influenced during runtime. From the Transcript, the system computes

$$(p_r(y), p_b(r)),$$

which may be left separate (Pareto analysis) or aggregated by user choice.

5.5 Optional Post-Processing

Once evaluation is complete, the researcher may optionally:

- generate plots (e.g. learning curve of accuracy vs. cost),
- compare multiple agents via Pareto frontiers,
- export transcripts for third-party analysis,
- publish reproducible Spec + Transcript bundles as benchmark assets.

This protocol transforms **The Causality Game** from a formal object into a **repeatable scientific experimentation process**, enabling principled comparison of domain-agnostic causal agents under strict experimental control.

In the next section, we instantiate this protocol with concrete **use cases and mission examples**.

6 Mission Examples and Use Cases

We now illustrate how **The Causality Game** accommodates fundamentally different causal inference tasks — all under the same formal game interface. Each task is encoded as a **MissionSpec**, which declares its objective and assigns appropriate p_r and p_b .

6.1 Mission Class 1: Causal Graph (DAG) Discovery

The agent must infer the correct directed causal structure underlying the SCM. The stopping action returns a predicted DAG $\hat{G}$. Typical result metrics include:

- **Structural Hamming Distance (SHD)**: number of edge additions/removals/reversals.
- **Edge-wise binary accuracy, F1, or AUROC** (directed or skeleton).

The behavioral metric may penalize number of rounds, interventions, or total sample count.

Objective. Minimize error in recovered structure at minimal interventional cost.

6.2 Mission Class 2: Treatment Effect Estimation (CATE / ATE / ITE)

The agent must estimate counterfactual treatment effects such as

$$\mathbb{E}[Y \mid \text{do}(T=1), X=x] - \mathbb{E}[Y \mid \text{do}(T=0), X=x].$$

Metric options include:

- **MSE, RMSE, PEHE**,
- **R²**,
- possible binary accuracy if reduced to uplift sign estimation.

If the agent ends early (few samples) yet produces a correct estimate, it is rewarded accordingly by p_b .

Objective. Maximize treatment effect accuracy while minimizing interventional burden.

6.3 Mission Class 3: Full SCM Reconstruction

The agent must learn either (i) the full structural equations f_i and noise distributions η_i , or (ii) sufficiently accurate approximations thereof for predictive equivalence.

Possible result metrics:

- **Functional similarity** via supervised regression metrics on held-out SCM samples,
- **KL divergence** between estimated vs. true noise,
- or their weighted convex combination.

Objective. Achieve $f_i^{\text{pred}} \approx f_i^{\text{true}}$ with minimal experimentation cost.

These examples illustrate that **The Causality Game** is not tied to any specific causal paradigm — it **generalizes across graph recovery, counterfactual estimation, structural discovery, and future tasks**.

In the next section, we present **results and experimental findings** obtained via this protocol.

7 Results

This section presents representative experimental results obtained using the **The Causality Game** framework — demonstrating its ability to evaluate agents under **multiple causal inference missions, different SCM families, and explicit interventional cost constraints**. Rather than committing to a single dataset or result type, we illustrate the breadth and rigor of the evaluation.

7.1 Evaluation Setting

Unless otherwise stated, each experiment consists of:

- one SCM instance drawn from a chosen **SCM generation source**,
- two or more **agents** — e.g., RandomAgent, ExhaustiveAgent, Active-Learning Agent,
- a fixed **mission type** (e.g., DAG recovery or CATE estimation),
- a hard or soft **interventional budget**,
- metrics defined via p_r (accuracy) and p_b (cost).

7.2 Example 1 — DAG Inference on Physical SCM

On a physical law-based SCM (e.g., $P = \frac{nRT}{V}$), the goal is to recover the *directed causal graph*. Figure ?? (previously displayed in Sec. 1) depicts:

- final **structural inference accuracy** vs. **total cost**,
- round-by-round **accuracy trajectory**,
- **Pareto-like tradeoff** among agents.

Active agents adaptively minimize cost; exhaustive agents maximize accuracy but spend more resources.

7.3 Example 2 — CATE Estimation under Limited Budget

In a semi-synthetic IHDP-derived SCM, the mission is to estimate CATE. The best agents learn to trade sample efficiency for statistical stability — sometimes **stopping early but optimally** under tight budget constraints. Figure ?? shows that budget-aware strategies achieve lower PEHE than naive baselines at comparable or lower intervention cost.

7.4 General Observations

Across all experiments:

- **interactive causality is fundamentally cost-sensitive**;
- blindly maximizing data collection is not Pareto-optimal;
- early stopping (with correct conclusions) is beneficial under p_b ;
- the **The Causality Game** framework reveals **agents’ reasoning quality**, not merely offline inference performance.

These insights highlight the necessity of interactive causal benchmarking — something fundamentally *not measured* in traditional static-dataset benchmarks.

We now discuss the broader implications of these findings, **limitations**, and promising **future research directions**.

8 Discussion, Limitations, and Future Work

9 Discussion

The Causality Game establishes a unified formalism for benchmarking *interactive causal inference*, evaluating agents not only on their final inference quality but also on their **efficiency in actively acquiring information**. This fundamentally departs from nearly all prior causal benchmarks, which provide agents with a fixed dataset and do not measure interventional strategy, adaptivity, or cost sensitivity.

The framework makes three key contributions:

1. It formalizes causal learning as a **resource-constrained, agent-driven game**.
2. It unifies diverse causal inference tasks (DAG recovery, CATE estimation, full SCM reconstruction) into a single abstraction.
3. It provides a fully seedable, serializable, and replayable environment — enabling fair, reproducible comparison of agents across research groups.

9.1 Current Limitations

Despite enabling a general and principled framework, **The Causality Game** currently does not yet aim to solve:

- **task-specific algorithm curation** — we do not claim to ship the best agents for each mission;
- **large-scale real-world SCM availability** — especially for domains where ground truth causality is unknowable or ethically inaccessible;
- **temporal and partially observable environments** — this initial formulation assumes stateless or snapshot-based SCMs;
- **domain realism guarantees** — although our framework supports realistic SCM injection (e.g. physical or dataset-based), we do not certify their real-world truthfulness.

9.2 Future Research Directions

We envision several high-value extensions:

- **Temporal and sequential SCMs** — incorporating dynamic causal processes and partially observable agents.
- **Advanced causal policies from foundation models** — testing large reasoning agents capable of planning interventions with memory and priors.
- **Shared curated benchmark suites** — collaborating with the research community to identify SCMs, datasets, or synthetic laws worthy of consensus benchmarking.
- **Multi-agent causal competitions** — extending **The Causality Game** into adversarial or cooperative causal inference settings.
- **Meta-learning and generalization tests** — ensuring agents generalize across families of SCMs, not just fit one.

The Causality Game provides the *infrastructure and scientific formalism* needed for the next generation of causal learning benchmarks — those that evaluate not only inference quality but also active experimental reasoning.